

ConwayLife Gui Alone con Pattori - Variante

Introduction

Goal: Realizzazione in Java dellla Gui [GAME OF LIFE DI CONWAY](#) con Pattori.

La logica del gioco è già stata analizzata nello [Sprint1](#). Utiliziamo quindi il risultato ottenuto, che costituisce un sottosistema funzionante, come punto di partenza per questo nuovo lavoro (deployed in un file **conway26Java-1.0.jar**). La costruzione, l'esecuzione e l'interazione di protoattori è stata realizzata con sviluppo del progetto [protoactor26](#). E' quindi disponibile la libreria **protoactor26-1.0.jar** che verrà integrata nel progetto.

Requirements

Realizzare una variante del progetto [conway26GuiAlonePactorTcp](#) nella ipotesi di costruire il sistema comprensivo sia di A che di S come un tutto organico, con le seguenti variazioni:

1. impostare i comandi cell e clear come request da S ad A
2. presentare, dopo la prima connessione, una pagina HTML alternativa priva di pulsanti
3. impostare l'adattamento di A a S attraverso un pattore local ad A che operi come Sidecar.

Requirement analysis

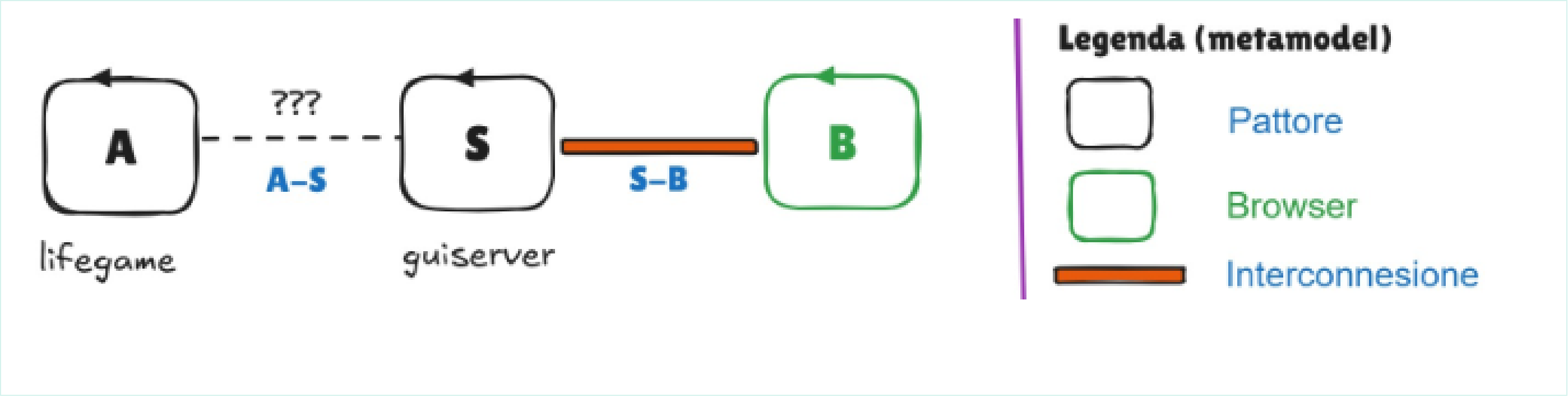
Il comando cell fa sì che S mandi un messaggio ad A per indurlo a modificare lo stato corrente di uana cella. Formalizzandolo come request (come da requisito R1), vincoliamo in modo implicito A ad inviare una risposta. Lo stesso ragionamento può essere fatto per il comando clear, che implica che A ponga lo stato di tutte le celle a morte. Dopo questa operazione A è tenuto a rispondere al messaggio.

Il secondo requisito comporta lo sviluppo di seconda pagina di output, da fornire a tutti i non-owner, che non presenterà i pultati START, STOP, CLEAR e EXIT. Inoltre, la pressione su celle non comporterà alcun comando inviato (togliamo la componente reattiva della pagina), ma la griglia dovrà essere aggiornata con l'evoluzione del gioco. Sarà compito del server tenere traccia della prima pagina che si è connessa e fornirle la pagina reattiva, dando invece la versione "ridotta" a tutti i successori.

Il terzo requisito parla di "pattern Sidecar". Il patter Sidecar è un modello di progettazione utilizzato nell'architettura software, in particolare negli ambienti basati su microservizi. In questo modello, un container o un processo “sidecar” viene implementato insieme al container dell'applicazione principale per estenderne o potenziarne le funzionalità. Il container sidecar opera nello stesso ambiente di esecuzione dell'applicazione principale e in genere fornisce servizi di supporto quali la registrazione degli eventi, il monitoraggio, la sicurezza o la comunicazione con altri servizi. Questo modello consente al container dell'applicazione principale di concentrarsi sulle proprie funzionalità principali, delegando al sidecar le attività secondarie (registrazione degli eventi, monitoraggio, sicurezza, individuazione dei servizi o proxy di comunicazione), favorendo così la modularità, la scalabilità e la manutenibilità nei sistemi distribuiti. Questo pattern viene utilizzato per evitare che la conoscenza del protocollo usato da S "inquinì" il componente A, senza utilizzare un proxy. E' una concreta realizzazione del pattern logico ACL (Anti-Corruption Layer): A introduce un modulo Adapter che conosce il protocollo di S e traduce le richieste di A nel formato richiesto da S. Quindi A parlerà a messaggi col Sidecar e il Sidecar parla con il servizio. Poiché un pattore può inviare messaggi messaggi ad altri pattori che fanno aprte dello stesso contesto, il Sidecar può essere un Pattore che fa aprte dello stesso contesto di A. Dobbiamo aggiungere un componente.

Problem analysis

Si può partire con l'analisi dalla seguente architettura logica schematizzata:



I componenti, alla luce di quanto evidenziato nell'analisi dei requisiti, sono:

- **A:** Pattore che rappresenta l'applicazione ConwayLife, contiene quindi la logica applicativa. Questo componente (a cui diamo il nome di lifegame) richiede l'uso di un'altro componente S per creare e gestire una GUI HTML, ma non lo conosce direttamente: interagisce solo ed esclusivamente con il suo Sidecar locale attraverso il protocollo di messaggi sviluppato con [IAplMessage](#).
- **Sidecar:** Pattore locale ad A, ovvero vive nel suo stesso contesto e che chiameremo lifeadapter. Funge da intermediario bidirezionale: riceve i dati della griglia da A e li traduce nel protocollo richiesto da S e riceve i messaggi da S, li interpreta e invia ad A comandi che può comprendere. Agisce quindi come un ponte, è il componente che garantisce il rispetto di DIP, come visto nell'analisi dei requisiti.
- **S:** Pattore che rappresenta un servizio di I/O mediante una pagina HTML. Questo componente (denominato guiservice) richiede l'uso di un componente B per visualizzare la pagina. Riceve messaggi con Sidecar e li inoltra a B e viceversa.
- **B:** Servizio che permette di visualizzare e di interagire con una pagina HTML.

Andiamo ora a formalizzare le interazioni tra i vari componenti. Per quanto riguarda le interazioni S-B sono basate su una interconnessione tra il componente S e il componente B, realizzata tramite protocollo HTTP/WS. Per quanto rigurada le interazioni A-S come abbiamo visto per rispettare il DIP viene inserito un sidecar.

Vengono formalizzati logicamente i vari messaggi scambiati.

A invia a Sidecar (la specifica del payload GRIDSTATE viene lasciata al progettista, in quanto dipende dalla rappresenazione della griglia):

- un dispatch per informare S dello stato corrente della griglia durante l'evoluzione del gioco (al temine di ogni Epoch), che formalizzo come:

```
msg(display, dispatch, lifegame, lifeadapter, GRIDSTATE, N)
```

- una reply alle request changecell e cmdclear di S, che formalizzo come:

```
msg(cellchanged, reply, lifegame, lifeadapter, cell(X,Y,done), N)
```

```
msg(doneclear, reply, lifegame, lifeadapter, GRIDSTATE, N)
```

Sidecar invia a S e A: inoltra i messaggi ricevuti, adattando il formato al protocollo del destinatario. **S invia a Sidecar** (si ricorda che non riteniamo opportuno che S conosca il significato dei comadi e quindi funge solo da tramite tra il browser e l'applicazione):

- una request per indurre A a modificare lo stato corrente di una cella, che formalizziamo come:

```
msg(changecell, request, guiservice, lifegame, cell(X,Y), N)
```

- una request per indurre A a resettare la griglia di gioco (comando clear), che formalizziamo come:

```
msg(cmdclear, request, guiservice, lifegame, clear, N)
```

- un dispatch per indurre A a fermare/riattivare il gioco o a interrompere la connessione, che formalizziamo come:

```
msg(cmd, dispatch, guiservice, lifegame, CMD, N) //CMD=start|stop|exit
```

S invia a B: S "parla" con la pagina mediante dei messaggi-stringa, inviando:

- un messaggio con l'indicazione del nome da attribuire alla pagina, appena questa si connette. Sta ad S fornire la pagina non reattiva ai non owner.
- un messaggio con l'indicazione del nuovo stato della griglia, che può essere granulare o globale (la definizione di questo dettaglio è lasciata al progettista).

B invia a S: La pagina deve "parlare" con il linguaggio (evoluto) del servizio S, inviando:

- un messaggio con l'indicazione del pulsante premuto (start|stop|clear|exit) premuto. Formalizzo con il seguente dispatch per start, stop e exit:

```
msg(cmd, dispatch, caller1, guiservice, CMD, N) //CMD=start|stop|exit
```

Formalizzo con la seguente request il comando clear:

```
msg(cmdclear, request, caller1, guiservice, clear, N)
```

- una request con l'indicazione della cella premuta dell'utente, che formalizzo nel seguente modo:

```
msg(changecell, request, caller1, guiservice, cell(X,Y), N)
```

S deve "girare" l'elaborazione di questi messaggi (cambiando mittente e destinatario) alla applicazione A.

Test plans

Project

Testing

Deployment

Maintenance